V

Viola's Cuisine

KITCHEN AS A CONCURRENT SYSTEM

A restaurant kitchen, **written in threads.**

A parallel-programming simulation in Python.
Bounded queues, a stove semaphore, a delivery condition variable, and a live Flask dashboard you can play with.

VIOLA ZENELI

Spring 2026

Why a kitchen is, secretly, **a concurrency problem.**

A real kitchen during service is the textbook concurrent system. Several waiters take orders at the same time. A small brigade of chefs is split across stations. Everyone wants the stoves. Some dishes take longer than others. And somebody still has to bring the food to the table.

I rebuilt that whole choreography in Python — **one thread per role** — and wrapped it in a live dashboard so I could **watch the synchronization primitives actually work**.

2 WAITERS
produce orders

5 CHEFS
across 3 stations

12 ORDERS
then the sim closes

All three are slider-controlled from the sidebar. These are just the values I'm walking through.

Four roles, one shared kitchen.

Every box is its own thread. Every arrow is a place where they have to agree.

WAITERS

PRODUCERS

Compose random orders from the menu

2 threads

▶

STATION QUEUES

BUFFERS

Grill · Pasta · Dessert
bounded, thread-safe

3 queue.Queue

▶

CHEFS

CONSUMERS

Acquire a stove,
cook, mark ready

5 threads

▶

DELIVERY

CONSUMER

Wakes on condition variable, serves table

1 thread

Shared resource

2 stoves, claimed by any grill or pasta chef before cooking. Whoever doesn't get one waits.

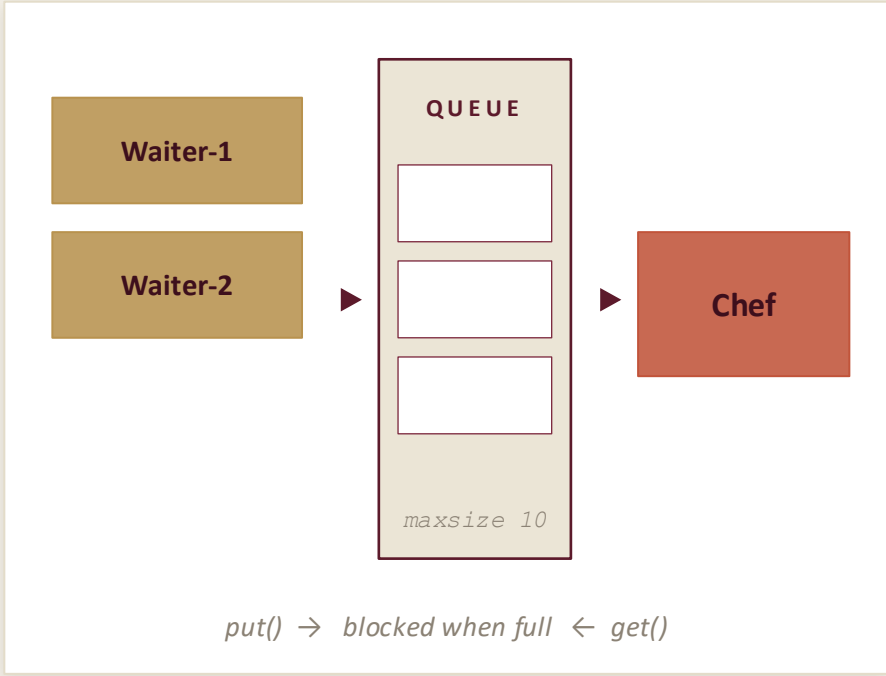
Sync point

Delivery sleeps on a condition variable. The chef who finishes the last dish wakes it up.

Producer & consumer

`queue.Queue`

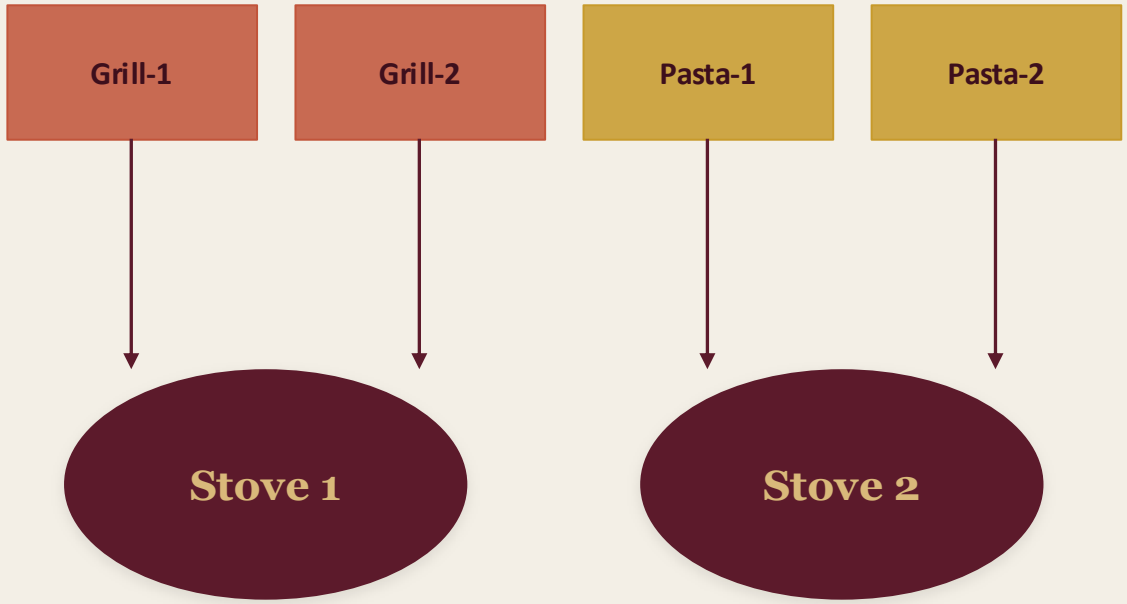
- Bounded buffer between the two sides.
- Waiters call put(); chefs call get(). Both block — coordination happens for free, no explicit lock.
- Three independent pipelines — one per station — so the grill is never stuck behind a dessert.
- Full queue → next put() blocks. That's back-pressure: the kitchen pushes back when it's overwhelmed.



Semaphore — the 2-stove rule

`threading.Semaphore`

All four want a stove. There are only two.



kitchen.py

```
self.stoves = Semaphore(2)

self.stoves.acquire()        # wait
try:
    cook(item)
finally:
    self.stoves.release()   # release
```

In a sentence: a semaphore is a counter that lets a fixed number of threads through at a time. It's how the simulation models a physical limit — a number of stoves, of phones, of database connections.

The two quieter primitives.

One protects shared memory. The other replaces polling with a wake-up.

MUTUAL EXCLUSION

threading.Lock

- Wraps the in-memory order book and the stats counters.
- Held only for a few lines at a time so nothing blocks long.
- Without it, two threads incrementing the served-orders counter at the same moment would lose updates — a classic race condition.

EVENT HAND-OFF

threading.Condition

- Delivery sleeps on it. No polling, no busy loop.
- Whenever a chef marks the last item of an order READY, it calls notify_all().
- Delivery wakes, picks up the order, serves the table, and goes back to sleep — instantly.

Random service times.

Why a fixed 4-second steak is a lie.

- Real services finish at different times. Two steaks rarely take exactly the same number of seconds — a constant time hides that.
- Queueing theory's classic answer is the exponential distribution: most dishes finish near the average, but a long tail of slow ones still happens.
- That long tail is what makes queues actually form — without it, the kitchen never feels under pressure.
- In code: random.expovariate(1 / mean_time), clamped so I never sample a 0-second steak or a 30-second one.
- Toggleable from the sidebar — flip it off to debug, flip it on to feel realistic.

waiter.py

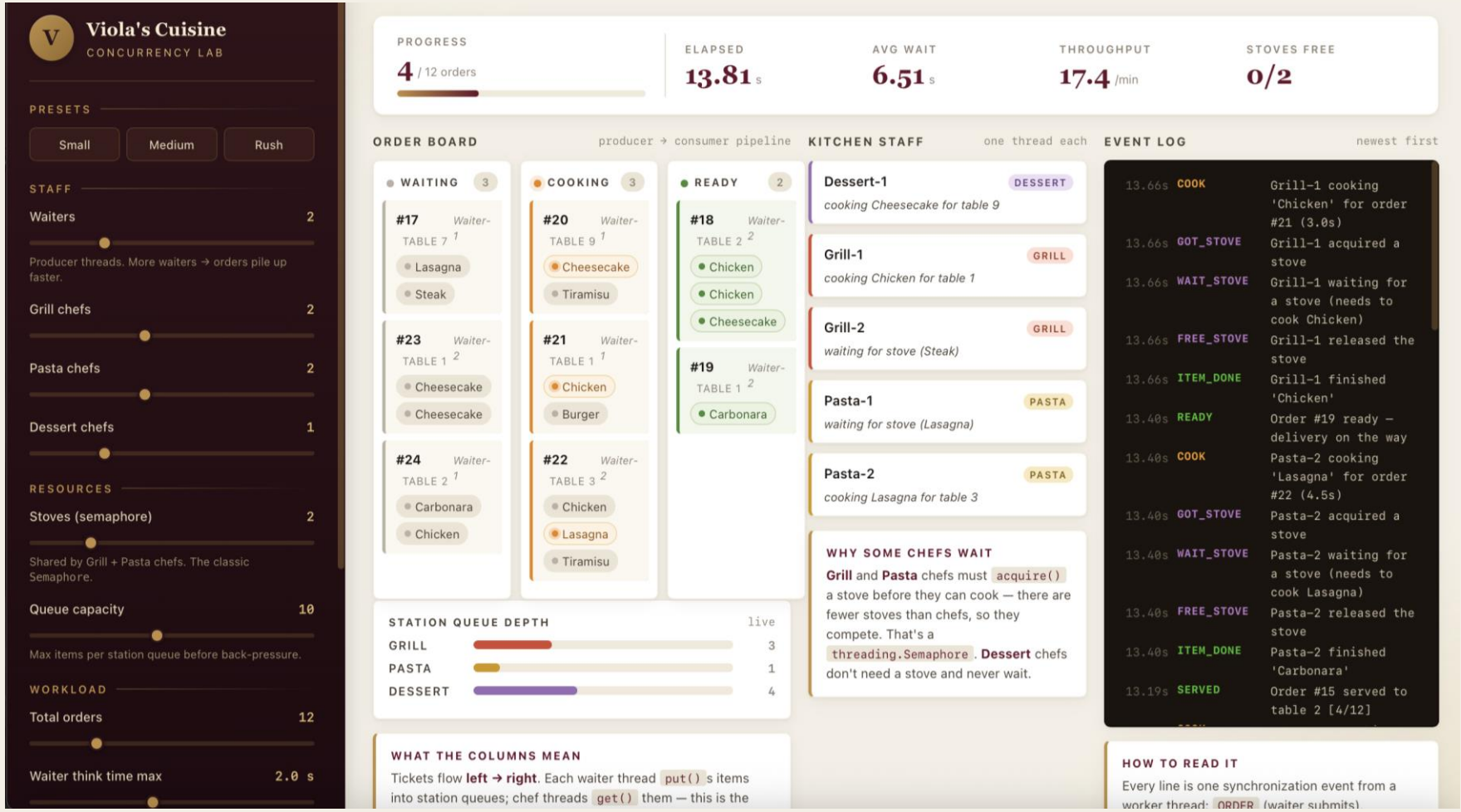
```
def sample(mean, rand):
    if not rand:
        return mean
    t = random.expovariate(1/mean)
    # long tail capped so no
    # 30-second steak sneaks in.
    return max(mean*0.4,
               min(mean*2.5, t))
```

Distribution of a 3-second dish under random_times = True:



Live in the browser.

Tweak the sidebar, hit Apply, watch the queues fill.



What you see

- **Sidebar**

  Every parameter is a slider — no editing source code.

- **Topbar**

  Progress, elapsed, average wait, throughput — all live.

- **Order board**

  Tickets march waiting → cooking → ready.

- **Kitchen staff**

  One card per chef thread; idle vs. cooking is colour-coded.

- **Queue depth**

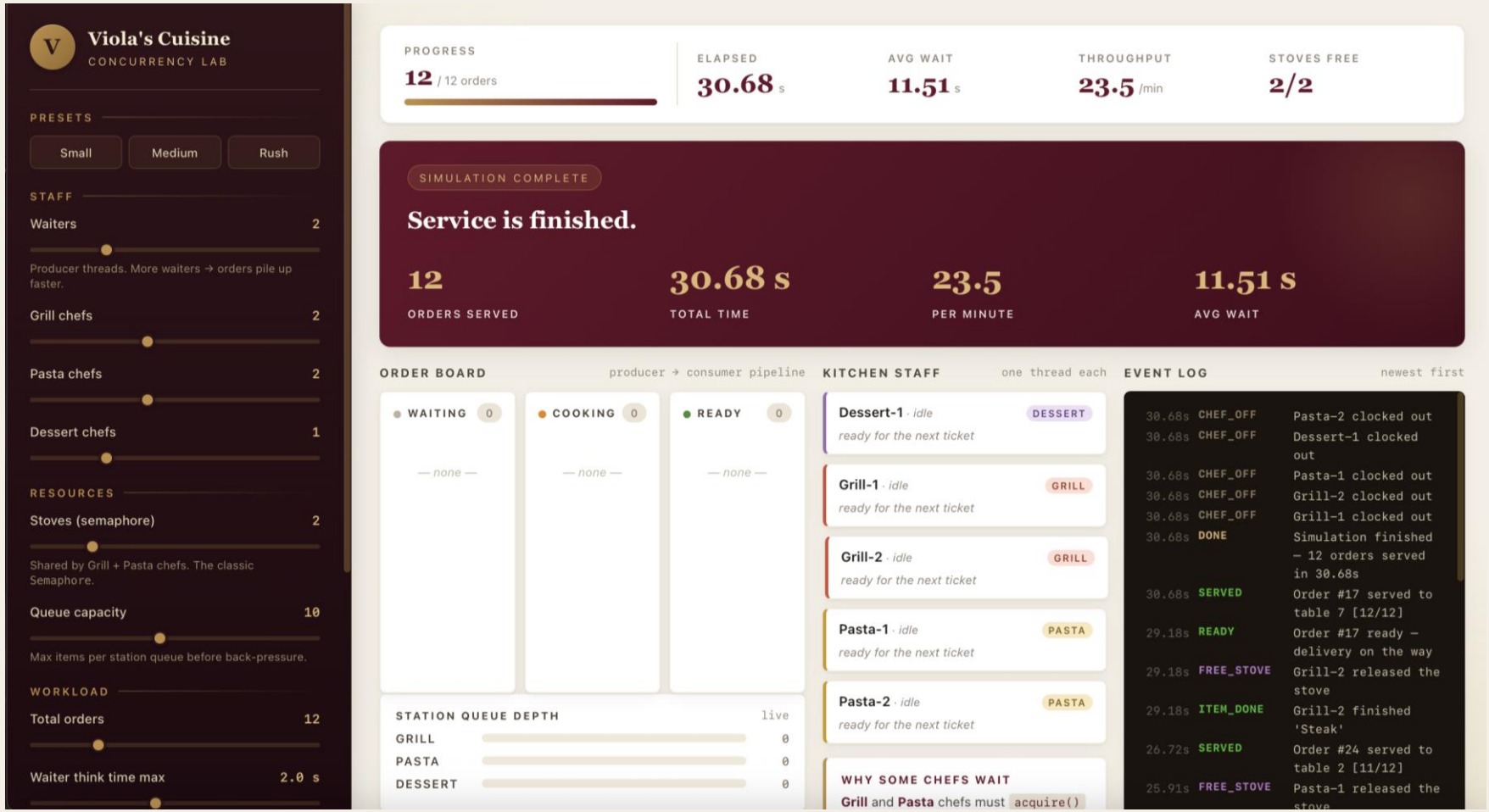
  Per-station fill bars show back-pressure visually.

- **Event log**

  Every sync event from every thread, newest first.

Every thread clocks out.

MAX_ORDERS reached → kitchen wakes everyone, threads exit cleanly, summary appears.



This particular run

12
orders served

30.68 s
total wall-clock

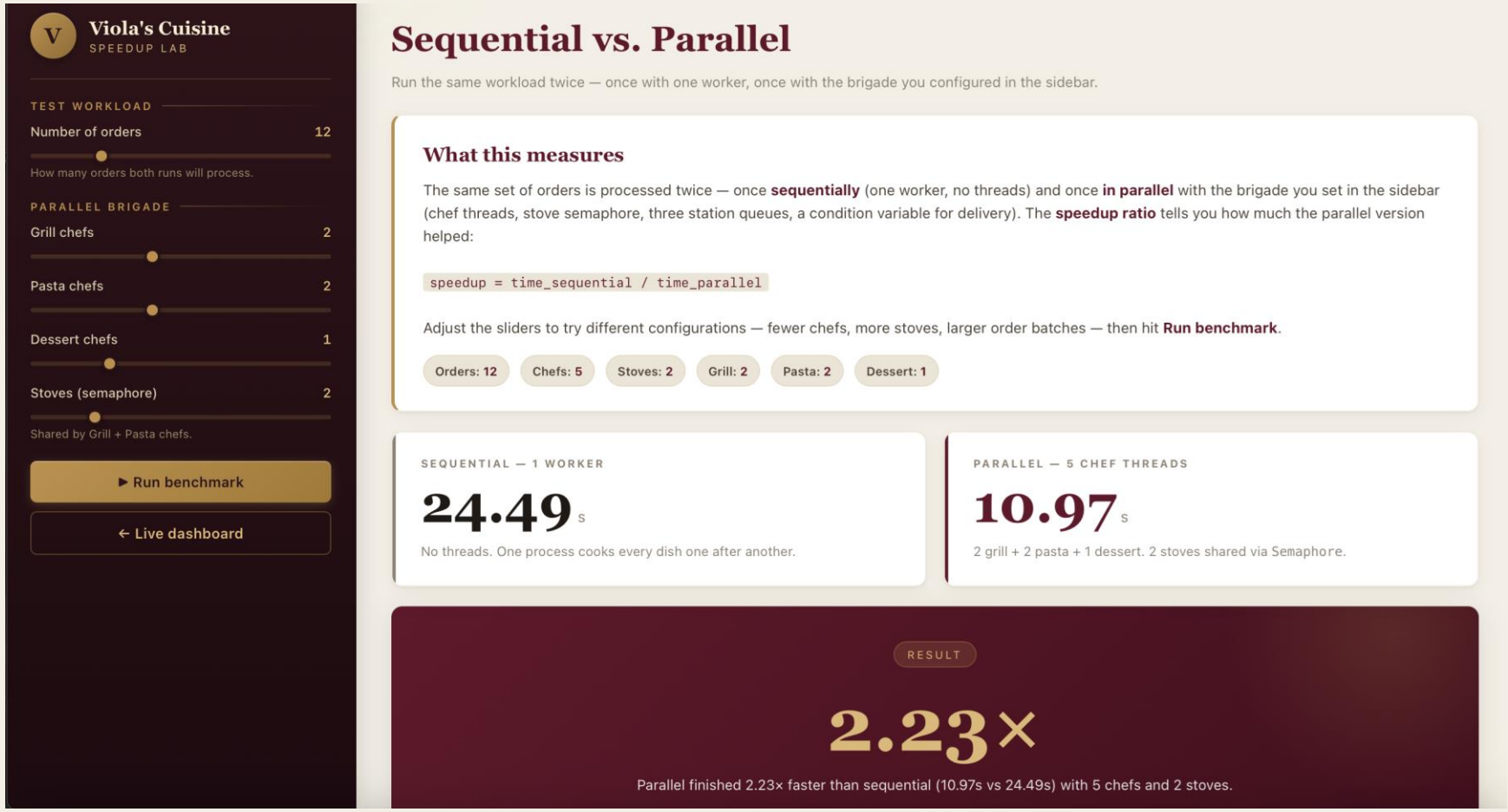
23.5
orders per minute

11.51 s
average wait per order

Restart from the sidebar — the dashboard picks up the new state on its next poll. No page reload.

How much did the threads actually help?

Same workload, run twice — once with one worker, once with the brigade you set.



This run

2.23×

PARALLEL / SEQUENTIAL

SEQUENTIAL	24.49 s
PARALLEL	10.97 s

Numbers change with the sidebar. Try fewer chefs, more stoves, larger batches.

If you want to try it yourself…

Five lines and a browser tab. Tested on macOS, Linux and Windows.

STEP 1 · CREATE A VIRTUAL ENV

Keeps the project's packages out of your system Python.

```
$ TERMINAL
python3 -m venv venv
```

STEP 2 · ACTIVATE IT

Pick the line that matches your machine.

```
$ TERMINAL
# macOS / Linux
source venv/bin/activate
# Windows (PowerShell):
venv\Scripts\Activate.ps1
```

STEP 3 · INSTALL THE DEPS

Flask is the only real dependency.

```
$ TERMINAL
pip install -r requirements.txt
```

STEP 4 · START THE KITCHEN

Threads spin up, the dashboard goes live on port 5000.

```
$ TERMINAL
python3 app.py
```

Then **`open http://127.0.0.1:5000`** in your browser, and play with the sidebar. **`Ctrl + C`** in the terminal stops everything cleanly.

V

Viola's Cuisine

FOUR PATTERNS · ONE KITCHEN

It's a kitchen
until it isn't.

Four primitives. Each one solves a different real coordination problem you can see, measure, and reason about — right in the dashboard.

Bounded queue | **Semaphore** | **Lock** | **Condition variable**

Thank you!